

Code Walkthrough

Table of Contents

代码逐函数讲解

结论

这份代码可以按“公共定义 -> PID -> 输入 -> 底盘 -> 云台 -> 调度 -> 硬件适配”的顺序理解。每个模块都遵守一个原则：只处理自己的职责，不跨模块乱改状态。

`robot_def.h`

作用：定义所有模块共享的类型和参数。

重点：

- `robot_mode_t`：整车 STOP/NORMAL。
- `chassis_mode_t`：独立、小陀螺、跟随云台。
- `remote_cmd_t`：遥控输入生成的统一命令。
- `Robot_LimitFloat()`：所有限幅都用它，避免输出超范围。
- `Robot_WrapAngleDeg()`：把 yaw 角误差限制到 $[-180, 180]$ 。

为什么这样写：公共定义集中后，后续调参数只改一个地方，不会在多个 .c 文件里找魔法数字。

`pid.c`

`PID_Init()`

设置 PID 参数、输出限幅、积分限幅，并清零内部状态。

`PID_Reset()`

清空积分和上次误差。急停时必须调用，否则恢复运行后可能因为积分残留导致电机突然冲一下。

PID_Calc()

计算逻辑：

```
error = target - feedback
integral += error * dt
derivative = (error - prev_error) / dt
output = kp * error + ki * integral + kd * derivative
```

为什么要积分限幅：防止长时间误差导致积分越积越大，恢复时输出失控。

remote.c

Remote_Init()

初始化按键状态和默认命令。默认进入 ROBOT_STOP，原因是上电默认停机更安全。

Button_Update()

内部函数，负责按键消抖、短按、长按、长按连发。

关键判断：

- 原始电平变化后先等待 REMOTE_DEBOUNCE_MS。
- 松开时如果没超过长按时间，判定为短按。
- 按住超过长按时间后，每隔 REMOTE_REPEAT_MS 产生一次连发事件。

Remote_Update()

把按键事件翻译成 remote_cmd_t：

- stop 短按切换 STOP/NORMAL。
- key1 短按切换小陀螺。
- key1 长按进入跟随云台。
- key2/key3 控制 yaw/pitch 目标角步进。

Remote_ParseUartFrame()

串口协议解析框架。它只负责把数据包解析成命令，不直接控制电机。

chassis.c

Chassis_Init()

初始化底盘状态和四个轮子的速度 PID。

`Chassis_CalcWheelTarget()`

把 $v_x/v_y/w_z$ 转成四个轮子的目标速度。这就是底盘运动学核心。

`Chassis_Control()`

控制流程：

1. 如果 STOP，调用 `Chassis_Stop()`。
2. 根据底盘模式修正 w_z 。
3. 计算四个目标轮速。
4. 读取四个编码器反馈。
5. 四路 PID 输出。
6. 输出限幅后发给电机。

`Chassis_Stop()`

目标速度、轮速输出、电机输出全部清零，并复位 PID。

`gimbal.c`

`Gimbal_Init()`

初始化 yaw/pitch 目标角和 PID 参数。

`Gimbal_Control()`

控制流程：

1. 如果 STOP，调用 `Gimbal_Stop()`。
2. 根据 key2/key3 更新目标角。
3. 对 pitch 目标角限位。
4. 读取 IMU yaw/pitch 角度。
5. 计算角度误差。
6. PID 输出云台电机控制量。
7. 输出限幅后发送给电机。

`Gimbal_Stop()`

清零云台输出并复位 PID。

app.c

App_Init()

初始化所有模块。

App_Loop()

主循环函数。它不直接写控制逻辑，只按时间调用 App_ControlTask()，同时更新 LED。

App_ControlTask()

控制任务核心：

```
Remote_Update(now_ms);  
Gimbal_Control(cmd, dt_s);  
Chassis_Control(cmd, gimbal->yaw_feedback_deg, dt_s);
```

为什么云台在底盘前面：底盘跟随云台模式需要用到最新 yaw 角反馈。

bsp_port.c

作用：把应用层和具体硬件隔离。

如果换开发板，只改这里：

- GPIO 按键
- LED
- PWM/CAN 电机输出
- 编码器反馈
- IMU 角度

为什么要隔离：控制算法不应该关心某个按键在哪个引脚，也不应该关心电机是 PWM 还是 CAN。

Source: E:/ai 产出文件/牛马/竞赛/竞

赛/outputs/robomaster/rm_infantry_control_learning_pack/Core/Src/app.c:49